CIS*6530 · Lecture 2 exercises: classification for security analytics

Estimated effort: 6 to 8 hours. **Not marked, no due date.** Data is in `data/lecture2/` plus your `features.parquet` from Lecture 1 Exercise 5 (or `reference_features.parquet` if you did not finish it: say which you used). Starter files are in `starter/lecture2/`. Check structure with `python tools/check_lecture2.py --out out`.

Fixed settings (so results are comparable)

- **Splits (log data):** train `hour < 2026-02-28`; gap 1 day; validation `2026-03-01 <= hour < 2026-03-07`; gap 1 day; test `hour >= 2026-03-08`. You may argue for a different gap in your report, but keep the cut dates.
- **Label:** `label = 1` for host-hours listed in `labels.csv` with `label = 1`; **all other host-hours are treated as benign** (an assumption to discuss).
- **Analyst budget:** `K = 15` host-hours reviewed per day for the triage exercise. You may choose another K if you justify it from a staffing argument.
- **Tune on validation. Open the test period once**, at the end. Report the number of configurations you tried.
- Fix random seeds. Every headline number gets a **95% interval from a bootstrap that resamples `src_ip` (hosts), not rows.**

Exercise 1: labels, splits, leakage

Output: `out/dataset.parquet`, `out/leakage_screen.csv`, `out/SPLITS.md`

1. Join `labels.csv` to your feature table on `(src_ip, hour)` with `validate="1:1"`. Report rows, positives, prevalence, and positives by `label_source`.
2. Add columns `label` (0/1 integer, unlabelled = 0), `label_source`, `label_ts`, and `split` (`train`, `val`, `test`, or `gap`).
3. Run the single-feature AUC screen on the **training** rows: `leakage_screen.csv` with columns `feature,auc,flagged`. A feature is flagged if `max(auc, 1-auc) > 0.95`. Then answer in `SPLITS.md`: for the highest-scoring feature, *could I have known this value at decision time?*
4. For positives only, report the share with `ioc_hit = 1` for each `label_source`. Explain what you see and why `ioc_hit` needs care as a model input.
5. Build a **host-grouped** 5-fold assignment (`StratifiedGroupKFold` on `src_ip`) and report positives per fold.

`SPLITS.md` must state the cut dates, positives per split, the gap and why, and how labels that arrive late (see `label_ts`) affect the rows just before each cut.

Exercise 2: models and a fair comparison

Output: `out/results.csv`, `out/importance.csv`

Compare, on the validation and test splits: the **prior** (constant score), your **Lecture 1 robust z-score**, your **Lecture 1 rarity score**, **logistic regression**, and **gradient boosting** (`HistGradientBoostingClassifier` or LightGBM). `results.csv` has one row per model and split with columns `model,split,ap,tpr_at_0_1pct_fpr,p_at_k,recall_at_k` (extra columns welcome). Also put `p_at_k_lo,p_at_k_hi` (the bootstrap interval) on the test rows.

- Preprocessing (scaling, `log1p`) goes inside a `Pipeline` and is fitted on training rows only.
- Handle imbalance with weighting or undersampling of the **training set only**.
- `importance.csv`: permutation importance of the boosted model on the validation set, scored with average precision (columns `feature,importance`). Comment on whether you believe the top features.

Report: which model is best on validation, whether that survives to test, and whether the intervals separate the models. A model that ties the rule is a legitimate finding. Do not hide it.

Exercise 3: triage at a budget

Output: `out/queue.csv`, `out/gain.png`, `out/calibration.png`, `out/TRIAGE.md`

1. Calibrate your best model on the validation rows (isotonic or sigmoid) and apply it to the test period.
2. `queue.csv`: for each test day, the top K host-hours to review, columns `day,rank,src_ip,hour,score,label`. Reserve about 10% of each day's K for a **random exploration slice**, and add a column `explore` (True/False).
3. `gain.png`: recall against the fraction of host-hours reviewed for your model and both Lecture 1 baselines, plus the random diagonal. `calibration.png`: a reliability curve (quantile bins) before and after calibration.
4. `TRIAGE.md`: precision at K and recall at K with intervals; the lift over random; how many host-hours must be reviewed to reach 80% recall for each method; and a short argument for your K (staff hours divided by minutes per alert).

Discuss: what the exploration slice tells you and costs you, and what would happen to future labels if you removed it. **Stretch:** rank by expected cost (calibrated probability times an impact weight from `crit`) instead of probability alone, and describe who wins.

Exercise 4: malware detection on EMBER-style features

Input: `malware_features.parquet` **Output:** `out/ember_report.md`, `out/malware_results.csv`, `out/malware_drift.png`

1. Train LightGBM on samples first seen **before 1 May 2025**. Evaluate on later samples: AP, TPR at 0.1% FPR, TPR at 1% FPR.
2. Compare three evaluations *with the same training-set size*: a **random** split, a **grouped split** (hold out whole `campaign`s), and the **time split** above. `malware_results.csv` columns: `split,ap,tpr_at_0_1pct_fpr,tpr_at_1pct_fpr`. Explain the differences.
3. Baseline: logistic regression on the `bytehist_*` columns only. What do the richer features buy?
4. **Drift plot:** TPR at 0.1% FPR for each test month (May to December) with the model fixed. State what the slope says about your retraining interval.
5. **Permutation importance by feature group** (byte histogram, byte entropy, strings, header, sections, imports), measured on the last four months.
6. Precision arithmetic: the file is about 50% malicious. Compute the precision you would get from your TPR and FPR at a real endpoint prevalence of 1% and of 0.1%.

Exercise 5: model card and pitfall audit

Output: `out/model_card.md` (or `.pdf`), two pages, use `starter/lecture2/model_card_template.md`

Purpose and decision supported; data and labels; splits; results **with intervals**; failure modes (include the malware drift and the post-8-March behaviour change in the log data); a deployment recommendation (**ship, run in shadow mode, or do not ship**) with reasons; and an audit against the ten pitfalls in Arp et al. (2022), naming the **two you are most exposed to** and citing evidence from your own work.

Quality bar

No leakage. Splits by time and entity. A baseline beside every model. An interval on every headline number. Raw data untouched. Fixed seeds. A README that reruns everything.

Stretch: simulate the feedback loop. For each test day, retrain on the labels revealed so far, show only the top K (with and without a 5% exploration slice), and compare how many incidents each policy ever surfaces.